

Dynamic Programming

All LCS

```
#include <bits/stdc++.h>
using namespace std;

// Build LCS length table C for strings a (length m) and b (length n).
vector<vector<int>> buildLCSMatrix(const string &a, const string &b) {
    int m = (int)a.size();
    int n = (int)b.size();
    vector<vector<int>> C(m + 1, vector<int>(n + 1, 0));
    for (int i = 1; i <= m; ++i) {
        for (int j = 1; j <= n; ++j) {
            if (a[i - 1] == b[j - 1])
                C[i][j] = C[i - 1][j - 1] + 1;
            else
                C[i][j] = max(C[i - 1][j], C[i][j - 1]);
        }
    }
    return C;
}

// Helper to create a unique key for memo map from (i,j).
inline long long keyFor(int i, int j) {
    return ((long long)i << 32) | (unsigned long long)j;
}

// backtrackAll returns a set of all distinct LCS strings for prefixes
// a[0..i-1], b[0..j-1]. C is the DP matrix (m+1 x n+1). Uses memoization to
// avoid repeated work.
set<string> backtrackAll(const vector<vector<int>> &C, const string &a,
                        const string &b, int i, int j,
                        unordered_map<long long, set<string>> &memo) {
    long long key = keyFor(i, j);
    auto it = memo.find(key);
    if (it != memo.end())
        return it->second;

    set<string> result;
    if (i == 0 || j == 0) {
        result.insert(string("")); // base: empty string
    } else if (a[i - 1] == b[j - 1]) {
        // characters match: append that char to all strings returned from (i-1,j-1)
        set<string> prev = backtrackAll(C, a, b, i - 1, j - 1, memo);
        for (const string &s : prev)
            result.insert(s + a[i - 1]);
    } else {
```

```

// follow both directions that are tied for optimal
if (C[i][j - 1] >= C[i - 1][j]) {
    set<string> s1 = backtrackAll(C, a, b, i, j - 1, memo);
    result.insert(s1.begin(), s1.end());
}
if (C[i - 1][j] >= C[i][j - 1]) {
    set<string> s2 = backtrackAll(C, a, b, i - 1, j, memo);
    result.insert(s2.begin(), s2.end());
}
}

memo.emplace(key, result);
return result;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string A, B;
    // Read two lines from stdin as the two strings.
    // (Use getline to allow spaces in the strings.)
    cout << "Enter first string (A):\n";
    if (!getline(cin, A))
        return 0;
    cout << "Enter second string (B):\n";
    if (!getline(cin, B))
        return 0;

    auto C = buildLCSMatrix(A, B);
    int m = (int)A.size();
    int n = (int)B.size();
    int lcs_len = C[m][n];

    cout << "\nLCS length = " << lcs_len << "\n";

    // If length is 0, one LCS is the empty string.
    if (lcs_len == 0) {
        cout << "Only LCS is the empty string.\n";
        return 0;
    }

    unordered_map<long long, set<string>> memo;
    set<string> all = backtrackAll(C, A, B, m, n, memo);

    // Filter out the empty string (shouldn't happen for positive length), and
    // only keep strings of length lcs_len.
    vector<string> results;
    results.reserve(all.size());
    for (const auto &s : all) {
        if ((int)s.size() == lcs_len)
            results.push_back(s);
    }
}

```

```
sort(results.begin(), results.end()); // ensure deterministic order
cout << "Number of distinct LCSs: " << results.size() << "\n\n";
cout << "All LCS strings (each on its own line):\n";
for (const auto &s : results)
    cout << s << "\n";

return 0;
}
```

Count Distinct Subsequences

```
// Given a string str of length n, your task is to find the count of distinct
// subsequences of it.
```

```
// Examples:
```

```
// Input: str = "gfg"
```

```
// Output: 7
```

```
// Explanation: The seven distinct subsequences are "", "g", "f", "gf", "fg",
// "gg" and "gfg"
```

```
// Input: str = "ggg"
```

```
// Output: 4
```

```
// Explanation: The four distinct subsequences are "", "g", "gg" and "ggg"
```

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
int distinctSubseq(string &str) {
```

```
    int n = str.size();
```

```
    // to store the last occurrence
```

```
    // of each character in the string
```

```
    vector<int> last(26, 0);
```

```
    // to store result after each index
```

```
    int res = 1;
```

```
    for (int i = 1; i <= n; i++) {
```

```
        // double the count of unique subsequences
```

```
        // and remove the repetition
```

```
        int cur = 2 * res - last[str[i - 1] - 'a'];
```

```
        // update the last occurrence of the character
```

```
        last[str[i - 1] - 'a'] = res;
```

```
        res = cur;
```

```
    }  
    return res;  
}
```

LCS - Count

```
#include <bits/stdc++.h>  
using namespace std;  
  
int lcs(string &s1, string &s2) {  
  
    int m = s1.length(), n = s2.length();  
  
    // dp vector is initialized to all zeros  
    // This vector stores the LCS values for the current row.  
    // dp[j] represents LCS of s1[0..i] and s2[0..j]  
    vector<int> dp(n + 1, 0);  
  
    // i and j represent the lengths of s1 and s2 respectively  
    for (int i = 1; i <= m; ++i) {  
  
        // prev stores the value from the previous  
        // row and previous column (i-1), (j -1)  
        // Used to keep track of LCS[i-1][j-1] while updating dp[j]  
        int prev = dp[0];  
  
        for (int j = 1; j <= n; ++j) {  
  
            // temp temporarily stores the current  
            // dp[j] before it gets updated  
            int temp = dp[j];  
  
            // If characters match, add 1 to the value  
            // from the previous row and previous column  
            // dp[j] = 1 + LCS[i-1][j-1]  
            if (s1[i - 1] == s2[j - 1])  
                dp[j] = 1 + prev;  
            else  
                // Otherwise, take the maximum of the  
                // left (dp[j-1]) and top (dp[j]) values  
                dp[j] = max(dp[j - 1], dp[j]);  
  
            // Update prev for the next iteration  
            // This keeps the value of the previous  
            // row (i-1) for future comparisons  
            prev = temp;  
        }  
    }  
}
```

```

    }

    // The last element of the vector contains the length of the LCS
    // dp[n] stores the length of LCS of s1[0..m] and s2[0..n]
    return dp[n];
}

```

LCS - Count of 3

```

#include <bits/stdc++.h>
using namespace std;

// Function to compute LCS of three strings using 2D DP (space optimized)
int lcsOf3(string &s1, string &s2, string &s3) {

    // Length of first string
    int n1 = s1.length();
    // Length of second string
    int n2 = s2.length();
    // Length of third string
    int n3 = s3.length();

    // Initialize two 2D arrays for DP:
    // prev holds values for previous i-1 level
    // curr holds values for current i level
    vector<vector<int>> prev(n2 + 1, vector<int>(n3 + 1, 0));
    vector<vector<int>> curr(n2 + 1, vector<int>(n3 + 1, 0));

    // Iterate over all characters of s1
    for (int i = 1; i <= n1; i++) {

        // Iterate over all characters of s2
        for (int j = 1; j <= n2; j++) {

            // Iterate over all characters of s3
            for (int k = 1; k <= n3; k++) {

                // If current characters of all three strings match
                if (s1[i - 1] == s2[j - 1] && s2[j - 1] == s3[k - 1])
                    curr[j][k] = 1 + prev[j - 1][k - 1];
                else
                    // Take the maximum of excluding current
                    // character from any one string
                    curr[j][k] = max({prev[j][k], curr[j - 1][k], curr[j][k - 1]});
            }
        }
        // Move curr to prev for the next iteration
    }
}

```

```
    prev = curr;
}

// The result is in curr[n2][n3], which holds the final LCS length
return curr[n2][n3];
}
```

Longest Common Substring - Count

```
#include <iostream>
#include <vector>
using namespace std;

// Given two strings 's1' and 's2', find the length of the longest common
// substring.
//
// Example:
//
// Input: s1 = "GeeksforGeeks", s2 = "GeeksQuiz"
// Output : 5
// Explanation:
// The longest common substring is "Geeks" and is of length 5.

int longestCommonSubstr(const string &s1, const string &s2) {
    int m = s1.length();
    int n = s2.length();

    // Create a 1D array to store the previous row's results
    vector<int> prev(n + 1, 0);

    int res = 0;
    for (int i = 1; i <= m; i++) {

        // Create a temporary array to store the current row
        vector<int> curr(n + 1, 0);
        for (int j = 1; j <= n; j++) {
            if (s1[i - 1] == s2[j - 1]) {
                curr[j] = prev[j - 1] + 1;
                res = max(res, curr[j]);
            } else {
                curr[j] = 0;
            }
        }
    }

    // Move the current row's data to the previous row
    prev = curr;
}
```

```
    return res;
}
```

Longest Common Supersequence - Count

```
// Given two strings s1 and s2, the task is to
// find the length of the shortest string
// that has both s1 and s2 as subsequences.

// Input: s1 = "geek", s2 = "eke"
// Output: 5
// Explanation: String "geeke" has both string "geek" and "eke" as subsequences.

#include <bits/stdc++.h>
using namespace std;

// Returns length of LCS for s1[0..m-1], s2[0..n-1]
int lcs(string &s1, string &s2) {
    int m = s1.size();
    int n = s2.size();

    // Initializing a matrix of size (m+1)*(n+1)
    vector<vector<int>> dp(m + 1, vector<int>(n + 1, 0));

    // Building dp[m+1][n+1] in bottom-up fashion
    for (int i = 1; i <= m; ++i) {
        for (int j = 1; j <= n; ++j) {
            if (s1[i - 1] == s2[j - 1])
                dp[i][j] = dp[i - 1][j - 1] + 1;
            else
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
        }
    }

    // dp[m][n] contains length of LCS for s1[0..m-1]
    // and s2[0..n-1]
    return dp[m][n];
}

int shortestCommonSupersequence(string &s1, string &s2) {

    return s1.size() + s2.size() - lcs(s1, s2);
}
```

Longest Palindromic Subsequence

```
#include <bits/stdc++.h>

// Given a string s, find the length of the Longest Palindromic Subsequence in
// it.

using namespace std;

// Function to find the length of the lps
int longestPalinSubseq(const string &s) {
    int n = s.size();

    // Create two vectors: one for the current state (dp)
    // and one for the previous state (dpPrev)
    vector<int> curr(n), prev(n);

    // Loop through the string in reverse (starting from the end)
    for (int i = n - 1; i >= 0; --i) {

        // Initialize the current state of dp
        curr[i] = 1;

        // Loop through the characters ahead of i
        for (int j = i + 1; j < n; ++j) {

            // If the characters at i and j are the same
            if (s[i] == s[j]) {

                // Add 2 to the length of the palindrome between them
                curr[j] = prev[j - 1] + 2;
            } else {

                // Take the maximum between excluding either i or j
                curr[j] = max(prev[j], curr[j - 1]);
            }
        }

        // Update previous to the current state of dp
        prev = curr;
    }

    return curr[n - 1];
}

int main() {
    string s = "bbabcbcab";
    cout << longestPalinSubseq(s);
    return 0;
}
```

Longest Palindromic Substring

// Given a string *s*, find the longest substring which is a palindrome. If there are multiple answers, then find the first appearing substring.

// Examples:

// Input: *s* = "forgeeksskeegfor"

// Output: "geeksskeeg"

// Explanation: The longest substring that reads the same forward and backward is "geeksskeeg". Other palindromes like "kssk" or "eeksskee" are shorter.

// Input: *s* = "Geeks"

// Output: "ee"

// Explanation: The substring "ee" is the longest palindromic part in "Geeks".

// All others are shorter single characters.

// Input: *s* = "abc"

// Output: "a"

// Explanation: No multi-letter palindromes exist. So the first character "a" is returned as the longest palindromic substring.

```
#include <iostream>
```

```
#include <string>
```

```
#include <vector>
```

```
using namespace std;
```

// Transforms the input string with sentinels and separators

```
string transformString(const string &s) {
```

```
    string ms = "@"; // left sentinel to avoid bounds check
```

```
    for (char c : s) {
```

```
        ms += "#" + string(1, c); // insert '#' between every character
```

```
    }
```

```
    ms += "$"; // right sentinel
```

```
    return ms;
```

```
}
```

// Runs Manacher's algorithm and returns the *p* array

```
vector<int> runManacher(const string &ms) {
```

```
    int n = ms.size();
```

```
    vector<int> p(n, 0);
```

```
    int l = 0, r = 0;
```

```
    for (int i = 1; i < n - 1; ++i) {
```

```
        // initialize p[i] based on its mirror and current [l, r] range
```

```
        if (r + 1 - i >= 0 && r + 1 - i < n)
```

```
            p[i] = max(0, min(r - i, p[r + 1 - i]));
```

```
        // try expanding around center i
```

```
        while (ms[i + 1 + p[i]] == ms[i - 1 - p[i]]) {
```

```

        ++p[i];
    }

    // update [l, r] if the new palindrome goes beyond current right boundary
    if (i + p[i] > r) {
        l = i - p[i];
        r = i + p[i];
    }
}
return p;
}

// Returns the radius of the longest palindrome centered at original index 'cen'
int getLongest(const vector<int> &p, int cen, int odd) {
    int pos = 2 * cen + 2 + !odd;
    return p[pos];
}

// Checks whether the substring s[l..r] is a palindrome
bool checkPalindrome(const vector<int> &p, int l, int r) {
    int res = getLongest(p, (r + l) / 2, (r - l + 1) % 2);
    return (r - l + 1) <= res;
}

// Finds and returns the longest palindromic substring in s
string getLongestPal(const string &s) {
    if (s.empty())
        return "";

    int n = s.size(), maxLen = 1, start = 0;

    // Transform the string and run Manacher's algorithm
    string ms = transformString(s);
    vector<int> p = runManacher(ms);

    for (int i = 0; i < n; ++i) {
        int oddLen = getLongest(p, i, 1);
        if (oddLen > maxLen) {
            // update start for odd-length palindrome
            start = i - (oddLen - 1) / 2;
            maxLen = oddLen;
        }

        int evenLen = getLongest(p, i, 0);
        if (evenLen > maxLen) {
            // update start for even-length palindrome
            start = i - (evenLen - 1) / 2;
            maxLen = evenLen;
        }
    }

    return s.substr(start, maxLen);
}

```

```
int main() {
    string s = "forgeeksskeegfor";
    cout << getLongestPal(s) << endl;
    return 0;
}
```

Longest Repeated Subsequence

```
// C++ program to find the longest repeating
// subsequence

// Given a string s, the task is to find the longest repeating subsequence, such
// that the two subsequences don't have the same string character at the same
// position, i.e. any ith character in the two subsequences shouldn't have the
// same index in the original string.
//
// Examples:
//
// Input: s = "aabb"
// Output: "ab"
// Explanation: The longest repeated subsequence is "ab", formed by matching 'a'
// from positions 1 and 2, and 'b' from positions 3 and 4.
//
// Input: s = "aab"
// Output: "a"
// Explanation: The two subsequences are 'a'(first) and 'a' (second). Note that
// 'b' cannot be considered as part of a subsequence as it would be at the same
// index in both.
//
// This problem is just the modification of the Longest Common Subsequence
// problem. The idea is to find the LCS(s, s) where s is the input string with
// the restriction that when both the characters are the same, they shouldn't be
// on the same index in the two strings.

#include <algorithm>
#include <iostream>
#include <string>
using namespace std;

int longestRepeatingSubsequence(string s) {
    int n = s.length();

    // Create and initialize DP table
    int dp[n + 1][n + 1];
    for (int i = 0; i <= n; i++)
        for (int j = 0; j <= n; j++)
            dp[i][j] = 0;
```

```

// Fill dp table (similar to LCS loops)
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= n; j++) {

        // If characters match and indexes are
        // not same
        if (s[i - 1] == s[j - 1] && i != j)
            dp[i][j] = 1 + dp[i - 1][j - 1];

        // If characters do not match
        else
            dp[i][j] = max(dp[i][j - 1], dp[i - 1][j]);
    }
}
return dp[n][n];
}

string longestRepeatedSubSeq(string s) {

    // THIS PART OF CODE IS
    // FILLS dp[][]
    int n = s.length();
    int dp[n + 1][n + 1];

    for (int i = 0; i <= n; i++) {
        for (int j = 0; j <= n; j++) {
            dp[i][j] = 0;
        }
    }

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            if (s[i - 1] == s[j - 1] && i != j) {
                dp[i][j] = 1 + dp[i - 1][j - 1];
            } else {
                dp[i][j] = max(dp[i][j - 1], dp[i - 1][j]);
            }
        }
    }

    // THIS PART OF CODE FINDS THE RESULT STRING USING DP[][]
    // Initialize result
    string res = "";

    // Traverse dp[][] from bottom right
    int i = n, j = n;
    while (i > 0 && j > 0) {

        // If this cell is same as diagonally
        // adjacent cell just above it, then
        // same characters are present at
        // str[i-1] and str[j-1]. Append any
        // of them to result.
    }
}

```

```

    if (dp[i][j] == dp[i - 1][j - 1] + 1) {
        res = res + s[i - 1];
        i--;
        j--;
    }

    // Otherwise we move to the side
    // that gave us maximum result
    else if (dp[i][j] == dp[i - 1][j]) {
        i--;
    } else {
        j--;
    }
}

// Since we traverse dp[][] from bottom,
// we get result in reverse order.
reverse(res.begin(), res.end());

return res;
}

```

Minimum insertions to form a palindrome

```

// Given a string s, the task is to find the minimum number of characters to be
// inserted to convert it to a palindrome.

// Input: s = "geeks"
// Output: 3
// Explanation: "skgeegks" is a palindromic string, which requires 3 insertions.

#include <bits/stdc++.h>
using namespace std;

// Function to find minimum insertions
// to make string palindrome
int findMinInsertions(string &s) {
    int n = s.size();
    vector<int> dp(n, 0);

    // Iterate over each character from right to left
    for (int l = n - 2; l >= 0; l--) {

        // This represents dp[l+1][h-1] from the previous row
        int prev = 0;
        for (int h = l + 1; h < n; h++) {

```

```
// Save current dp[h] before overwriting
int temp = dp[h];

if (s[l] == s[h]) {

    // No new insertion needed if characters match
    dp[h] = prev;
} else {

    // Take min of two cases + 1
    dp[h] = min(dp[h], dp[h - 1]) + 1;
}

// Update prev for the next column
prev = temp;
}
}

return dp[n - 1];
}
```

Optimal Edit Distance

```
// C++ program to find minimum number
// of operations to convert s1 to s2
#include <bits/stdc++.h>
using namespace std;

int editDistance(string &s1, string &s2) {
    int m = s1.size();
    int n = s2.size();

    int prev;
    vector<int> curr(n + 1, 0);

    for (int j = 0; j <= n; j++)
        curr[j] = j;

    for (int i = 1; i <= m; i++) {
        prev = curr[0];
        curr[0] = i;
        for (int j = 1; j <= n; j++) {
            int temp = curr[j];
            if (s1[i - 1] == s2[j - 1])
                curr[j] = prev;
            else
                curr[j] = 1 + min({curr[j - 1], prev, curr[j]});
        }
    }
}
```

```
        prev = temp;
    }
}

return curr[n];
}
```

Optimal Edit Distance - Without Edit

```
#include <bits/stdc++.h>
using namespace std;

int lcs(string &s1, string &s2) {
    int m = s1.length(), n = s2.length();
    vector<vector<int>> dp(2, vector<int>(n + 1));

    for (int i = 0; i <= m; i++) {
        bool curr = i & 1;

        for (int j = 0; j <= n; j++) {
            if (i == 0 || j == 0)
                dp[curr][j] = 0;

            else if (s1[i - 1] == s2[j - 1])
                dp[curr][j] = dp[1 - curr][j - 1] + 1;

            else
                dp[curr][j] = max(dp[1 - curr][j], dp[curr][j - 1]);
        }
    }

    return dp[m & 1][n];
}

int minOperations(string s1, string s2) {
    int m = s1.size();
    int n = s2.size();

    // the length of the LCS for s1[0..m-1] and s2[0..n-1]
    int len = lcs(s1, s2);

    // Characters to delete from s1
    int minDeletions = m - len;
}
```

```
// Characters to insert into s1
int minInsertions = n - len;

// Total operations needed
int total = minDeletions + minInsertions;
return total;
}
```

Optimal Knapsack

```
int KnapSack(int val[], int wt[], int n, int W) {
    // array to store final result
    // dp[i] stores the profit with KnapSack capacity "i"
    int dp[W + 1];

    // initially profit with 0 to W KnapSack capacity is 0
    memset(dp, 0, sizeof(dp));

    // iterate through all items
    for (int i = 0; i < n; i++)
        // traverse dp array from right to left
        for (int j = W; j >= wt[i]; j--)
            dp[j] = max(dp[j], val[i] + dp[j - wt[i]]);
    /*above line finds out maximum of dp[j](excluding ith
    element value) and val[i] + dp[j-wt[i]] (including ith
    element value and the
    profit with "KnapSack capacity - ith element weight")
    */
    return dp[W];
}
```

Perfect Sum

```
// Given an array arr[] of non-negative integers and an integer target. The task
// is to print all subsets of the array whose sum is equal to the given target.
// Note: If no subset has a sum equal to target, print -1.

// Examples:
```

```
// Input: arr[] = [5, 2, 3, 10, 6, 8], target = 10
// Output: [ [5, 2, 3], [2, 8], [10] ]
// Explanation: We need to find all subsets of arr[] that sum up to target = 10.
// Subset [5, 2, 3] - Sum = 5 + 2 + 3 = 10
// Subset [2, 8] - Sum = 2 + 8 = 10
// Subset [10] - Sum = 10

// Input: arr[] = [5, 7, 8], target = 3
// Output: [ [-1] ]
// Explanation: There are no subsets of the array that sum up to the target 3.

// Input: arr[] = [35, 2, 8, 22], target = 0
// Output: [ [ ] ]
// Explanation: The empty subset is the only subset with a sum of 0.
```

```
#include <bits/stdc++.h>
using namespace std;
```

```
// Recursively finds all subsets with the given target
void findSubsets(vector<int> &arr, vector<vector<bool>> &dp, int i, int target,
                vector<int> &curr, vector<vector<int>> &res) {
```

```
    // Base case: If target becomes 0
    if (i == 0) {
        if (target == 0)
            res.push_back(curr);
        if (arr[0] == target) {
            curr.push_back(arr[0]);
            res.push_back(curr);
            curr.pop_back();
        }
        return;
    }

    // Exclude current element
    if (dp[i - 1][target]) {
        findSubsets(arr, dp, i - 1, target, curr, res);
    }

    // Include current element if it does not exceed target
    if (target >= arr[i] && dp[i - 1][target - arr[i]]) {
        curr.push_back(arr[i]);
        findSubsets(arr, dp, i - 1, target - arr[i], curr, res);
        curr.pop_back();
    }
}
```

```
// Returns all subsets with the given target sum
vector<vector<int>> perfectSum(vector<int> &arr, int target) {
    int n = arr.size();
    if (n == 0 || target < 0)
        return {};

```

```
    // DP table to store subset sum possibilities
```

```
vector<vector<bool>> dp(n, vector<bool>(target + 1, false));

// Correct DP initialization for handling zeroes
dp[0][0] = true;
if (arr[0] <= target)
    dp[0][arr[0]] = true;

for (int i = 1; i < n; ++i) {
    for (int j = 0; j <= target; ++j) {
        dp[i][j] = dp[i - 1][j] || (arr[i] <= j && dp[i - 1][j - arr[i]]);
    }
}

// If no subsets sum to target, return empty
if (!dp[n - 1][target])
    return {};

vector<vector<int>> res;
vector<int> curr;
findSubsets(arr, dp, n - 1, target, curr, res);
return res;
}

// Function to print subsets in required format
void print2dArray(vector<vector<int>> &arr) {
    if (arr.empty()) {

        // No valid subsets found
        cout << "-1\n";
        return;
    }

    // Printing subsets in formatted output
    for (int row = 0; row < arr.size(); row++) {
        cout << "[";
        for (int col = arr[row].size() - 1; col >= 0; col--) {
            cout << arr[row][col];
            if (col != 0) {
                cout << ", ";
            }
        }
        cout << "];";
        if (row < arr.size() - 1)
            cout << ", ";
    }
}

// Driver function
int main() {
    vector<int> arr = {5, 2, 3, 10, 6, 8};
    int target = 10;
    vector<vector<int>> result = perfectSum(arr, target);
    print2dArray(result);
}
```

```
    return 0;
}
```

Regex Matching

```
#include <iostream>
#include <vector>
using namespace std;

bool isMatch(string t, string p) {
    int n = t.size();
    int m = p.size();

    // DP table where dp[i][j] means whether first i characters in t
    // match the first j characters in p
    vector<vector<bool>> dp(n + 1, vector<bool>(m + 1, false));

    // Empty pattern matches empty text
    dp[0][0] = true;

    // Deals with patterns like a*, a*b*, a*b*c* etc, where '*'
    // can eliminate preceding character
    for (int j = 1; j <= m; ++j) {
        if (p[j - 1] == '*' && j > 1) {
            dp[0][j] = dp[0][j - 2];
        }
    }

    // Fill the table
    for (int i = 1; i <= n; ++i) {
        for (int j = 1; j <= m; ++j) {

            // Characters match
            if (p[j - 1] == '.' || t[i - 1] == p[j - 1]) {
                dp[i][j] = dp[i - 1][j - 1];
            }

            else if (p[j - 1] == '*' && j > 1) {

                // Two cases:
                // 1. '*' represents zero occurrence of the preceding character
                // 2. '*' represents one or more occurrence of the preceding character
                dp[i][j] = dp[i][j - 2] ||
                    (dp[i - 1][j] && (p[j - 2] == t[i - 1] || p[j - 2] == '.'));
            }
        }
    }
}
```

```

    return dp[n][m];
}

int main() {
    cout << boolalpha << isMatch("aab", "a.*") << endl;
    cout << boolalpha << isMatch("aa", "a") << endl;
    cout << boolalpha << isMatch("aa", "a*") << endl;
    cout << boolalpha << isMatch("ab", ".") << endl;
    cout << boolalpha << isMatch("mississippi", "mis*is*p*.") << endl;
    return 0;
}

```

Subset Sum - Count

```

// C++ Program for Space Optimized Dynamic Programming
// Solution to Subset Sum Problem
#include <bits/stdc++.h>
using namespace std;

// Returns true if there is a subset of arr[]
// with sum equal to given sum
bool isSubsetSum(vector<int> arr, int sum) {
    int n = arr.size();
    vector<bool> prev(sum + 1, false), curr(sum + 1);

    // Mark prev[0] = true as it is true
    // to make sum = 0 using 0 elements
    prev[0] = true;

    // Fill the subset table in
    // bottom up manner
    for (int i = 1; i <= n; i++) {
        for (int j = 0; j <= sum; j++) {
            if (j < arr[i - 1])
                curr[j] = prev[j];
            else
                curr[j] = (prev[j] || prev[j - arr[i - 1]]);
        }
        prev = curr;
    }
    return prev[sum];
}

// Returns the count of subsets of arr[] with sum equal to given sum
int countSubsetSum(vector<int> arr, int sum) {
    int n = arr.size();

```

```
vector<int> prev(sum + 1, 0), curr(sum + 1);

// Base case: there's 1 way to make sum = 0 (empty subset)
prev[0] = 1;

for (int i = 1; i <= n; i++) {
    for (int j = 0; j <= sum; j++) {
        if (j < arr[i - 1]) {
            curr[j] = prev[j];
        } else {
            curr[j] = prev[j] + prev[j - arr[i - 1]];
        }
    }
    prev = curr;
}
return prev[sum];
}
```

Subset Sums

```
#include <bits/stdc++.h>
using namespace std;

// Prints sums of all subsets of array
void subsetSums(vector<int> &nums, int n) {
    // There are total 2^n subsets
    vector<int> s = {0}; // store the sums

    for (int i = 0; i < n; i++) {
        const int v = s.size();
        for (int t = 0; t < v; t++) {
            s.push_back(s[t] + nums[i]); // add this element with previous subsets
        }
    }
    // Print
    for (int i = 0; i < s.size(); i++)
        cout << s[i] << " ";
}
```